Beyond F1: A Comparative Study of LLM-Based Identifier Renaming for Automatically Generated Tests

Jason Liu
Master Student Software Engineering
University of Antwerp
Antwerp, Belgium
jason.liu@student.uantwerpen.be

***Abstract*—Automated test generation tools such as EvoSuite and Randoop produce tests that are functionally correct but nearly unreadable — method names like `func_1` and variables like `var_1` force developers to trace through the entire test body before they can understand, trust, or maintain what they are looking at. Renaming these identifiers is therefore essential, but it is unclear which approach does it best. A model specifically trained on renamed test code produces more accurate names than any other approach we compared — yet its tests are consistently less readable than those produced by a model that has never seen the task before. Naming accuracy and readability measure fundamentally different things, and evaluating only one gives a misleading picture of which approach actually helps a developer.**

***Index Terms*—Large Language Models, Automated test generation, Identifier renaming, Test readability, Fine-tuning, LLM-as-a-Judge, Unit tests, Java, Code comprehension**

I. Introduction

Automated test generation tools such as EvoSuite [1] and Randoop [2] are widely used to produce test suites that achieve high code coverage with minimal manual effort. The tests they produce are functionally correct — they exercise the code and catch regressions — but they are written for a machine, not a developer. Method names like `func_1` and variables like `var_1` force anyone reading the test to trace through the entire body before understanding what scenario is being tested, what values are involved, and whether the assertions make sense. This makes generated tests hard to inspect, hard to maintain, and hard to trust [3], [4].

Renaming the identifiers is the natural solution, and several approaches have been proposed — from graph neural networks that reason about code structure [5], [6] to large language models (LLMs) that leverage broad knowledge of naming conventions [7]. But it is not yet clear which approach produces the best results, nor how to measure "best" — because, as we show in Section II, multiple valid names exist for the same variable, meaning a name can be correct without matching any single expected answer.

This paper presents a comparative study of LLM-based and non-LLM-based approaches to identifier renaming in automatically generated tests, evaluated on both naming accuracy and holistic readability. We compare two non-LLM baselines (GGNN and RefBERT), three LLMs used without task-specific training (GPT-4.1 mini, GPT-5 mini and Qwen2.5-Coder), and a locally deployed open-source model (Qwen2.5-Coder) fine-tuned at five stages of training. To make readability evaluation scalable without requiring a human study, we contribute CodeReader — a configurable open-source tool that uses an ensemble of LLM judges to score test readability against an explicit rubric.

Our results show that LLMs substantially outperform non-LLM approaches on naming accuracy, but reveal a striking divergence: the fine-tuned model that scores highest on accuracy produces less readable tests than models that were never trained on the task. This suggests that naming accuracy and readability are not interchangeable measures of renaming quality, and that future work should evaluate both.

The rest of the paper is structured as follows. Section II presents a motivating example that illustrates the core tension between accuracy and readability. Section III provides the background needed to understand the study. Section IV surveys prior work and identifies the gaps this study addresses. Section V describes the study design, pipeline, and evaluation metrics. Section VI presents the results. Section VII discusses their implications and threats to validity. Section VIII at last concludes the paper and all the findings that we found.

II. Motivating Example

To see why identifier renaming in generated tests is both important and difficult to evaluate, consider a concrete case. We take a human-written test for `DateUtils.yearEnd()` and replace its identifiers with generic placeholders - the same style of names that tools like EvoSuite and Randoop produce automatically - yielding the following:

```
@Test public void func_1() {
    Date var_1 = DateUtils.yearEnd();
    assertNotNull(var_1);
            GregorianCalendar var_2 = new
GregorianCalendar();
    var_2.setTime(var_1);
    assertEquals(12, var_2.get(MONTH));
    assertEquals(31, var_2.get(DAY_OF_MONTH));
}
```

Listing 1. Obfuscated test for `DateUtils.yearEnd()`: original identifiers replaced with generic placeholders to simulate auto-generated output.

The test is correct — it verifies that `DateUtils.yearEnd()` returns a date falling on December 31 — but a developer reading it for the first time learns nothing from the names `func_1`, `var_1`, or `var_2`. They must read the entire body, trace each variable through its uses, and mentally reconstruct the intent before they can trust, debug, or extend the test. This is the problem that identifier renaming aims to solve.

A. A first attempt at renaming

A human developer writing this test from scratch would produce something closer to the oracle (the human-written reference name) version shown in Listing 2. The method is named after the production method it exercises, and the two variables are named after the values they hold.

```
@Test public void yearEnd() {
    Date date = DateUtils.yearEnd();
    assertNotNull(date);
          GregorianCalendar calendar = new
GregorianCalendar();
    calendar.setTime(date);
    assertEquals(12, calendar.get(MONTH));
   assertEquals(31, calendar.get(DAY_OF_MONTH));
}
```

Listing 2. Oracle (human-written) version of the same test.

The names are short, consistent with the surrounding codebase's conventions, and immediately informative. A natural approach to automating this is therefore to train a model on pairs of obfuscated - replacing meaningful variable and function names with meaningless symbols - and human-written tests and measure how closely its output matches the oracle. The standard metric for this is the sub-token-level F1 score, which counts how many of the name parts (sub-tokens) the model predicts correctly. On this test, for example, the oracle method name `yearEnd` has two sub-tokens: `year` and `end`. A prediction of `testYearEnd` would score 1.0 recall (both sub-tokens present) but lower precision (it adds `test`), yielding a partial F1 score.

B. The multiple-names problem

The difficulty is that there is no single correct name for a given identifier. Table I shows the names produced by three different LLM-based renaming approaches for the same obfuscated test, alongside the oracle. All three models see only the test body — they have no access to the source code of `DateUtils.yearEnd()`.

TABLE I
NAMES PRODUCED BY FOUR SYSTEMS FOR THE THREE IDENTIFIERS IN LISTING 1. ALL LLM OUTPUTS ARE PLAUSIBLE; NONE MATCHES THE ORACLE EXACTLY.

System	Method name	`var_1` (Date)	`var_2` (Calendar)
Oracle	yearEnd	date	calendar
Qwen2.5-Coder	testShouldReturn YearEndMonthAndDay	yearEndDate	actualCalendar
GPT-4.1 mini	testYearEnd ReturnsLastDayOfYear	actualYear EndDate	calendar
GPT-5 mini	testYearEnd ReturnsDecember31	actualYear EndDate	yearEnd Calendar

Each output is a reasonable name. `actualDate`, `date`, and `actualYearEndDate` all correctly describe a `Date` variable holding the return value of `yearEnd()`. `calendar`, `actualCalendar`, and `yearEndCalendar` all correctly describe the `GregorianCalendar` used to inspect that date. Yet scored against the oracle, almost every LLM output receives a partial F1 at best.

This reveals a fundamental limitation of reference-based evaluation - where we score a prediction by comparing it against a single expected answer: a correct rename can be penalized simply for being more descriptive than the oracle chose to be. A model that learns to reproduce the concise oracle style will score well on F1 — but conciseness and readability are not the same thing. A developer encountering `testShouldReturnYearEndMonthAndDay` for the first time understands the test's intent immediately; a developer encountering `yearEnd` must still read the body to know what is being asserted.

III. Background

This section provides the background necessary to understand our approach, covering automated test generation, identifier quality, obfuscation, sub-token-level f1, LLM-based evaluation, large language models for code, fine-tuning and dataset.

A. Automated Test Generation

EvoSuite [1] evolves a population of test cases using a genetic algorithm that maximises a coverage criterion such as branch coverage. Randoop [2] generates sequences of method calls incrementally, using execution feedback to discard failing sequences and retain those that extend coverage. Both tools operate purely on structural and execution properties, with no awareness of semantic meaning, so the identifiers they emit are generic and uninformative: local variables receive names such as `double0` or `calculator0`, and test methods receive names such as `test0` or `test42` [8], [9]. Examples are shown in Appendix D.1 and Appendix D.2.

B. Identifier quality and test readability

A high-quality identifier satisfies three criteria [6]: *consistency* (no synonyms or homonyms across the codebase), *correctness* (the name corresponds to the concept it represents), and *conciseness* (the name is no more general than the concept

warrants). Butler et al. demonstrated empirically that poor identifier naming degrades a developer's ability to understand and maintain code [10], and Lin et al. found that test code identifiers are substantially lower quality than production code identifiers, with only 61.6% rated acceptable compared to 92.5% for production code [11]. Test method names carry an additional communicative burden: the established JUnit convention encodes the scenario and expected outcome directly in the name using a pattern such as `methodName_stateUnderTest_expectedBehavior` [3]. When generated tests use names like `test42`, neither purpose is served.

Of the three criteria, *consistency* occupies a special position. A name that is individually readable but inconsistent with the surrounding codebase's naming conventions may be harder to work with than a generic name that follows a predictable pattern, because developers rely on naming regularity to navigate and reason about code quickly [10], [12]. Winkler et al.'s systematic mapping study confirmed that identifier consistency is among the primary factors influencing comprehension of test code, and that no single metric adequately captures all dimensions of test readability [13].

C. Obfuscation

Automated test generation tools such as EvoSuite and Randoop produce tests with generic, meaningless identifiers because they optimize for code coverage, not readability. To simulate this in a controlled way, we take human-written tests — where the original identifiers are meaningful — and systematically replace every identifier with a generic placeholder: test methods become `func_n` and local variables become `var_n`, where `n` is a counter. This process is called *obfuscation*. Crucially, only the names are changed; the structure, logic, and assertions of the test remain identical. This means we know the correct names (the originals), giving us a ground truth to measure against.

D. Subtoken-level F1

When evaluating how closely a predicted name matches the oracle, we use the *sub-token-level F1 score*. Identifiers in Java follow camel case conventions, so a name like `yearEndDate` is naturally split into three sub-tokens: `year`, `end`, and `date`. F1 is the harmonic mean of precision and recall over these sub-tokens.

To make this concrete, consider the oracle name `yearEnd` — split into sub-tokens `year` and `end`. If a model predicts `testYearEnd`, the sub-tokens are `test`, `year`, and `end`.

- *Recall* measures how many oracle sub-tokens the prediction contains: both `year` and `end` are present, so recall = 2/2 = 1.0.
- *Precision* measures how many predicted sub-tokens are in the oracle: `year` and `end` match, but `test` does not, so precision = 2/3 = 0.67.
- *F1* is then 2 × (0.67 × 1.0) / (0.67 + 1.0) = **0.80**.

The F1 score is computed per identifier and averaged across all identifiers in the test suite. A score of 1.0 means every predicted sub-token matches the oracle exactly; a score of 0.0 means no overlap at all.

This means that `testYearEndReturnsDecember31` — a name that tells the reader exactly what the test will return — scores **0.50** against the oracle `yearEnd`, simply because it is more descriptive. This is the limitation that motivates looking beyond F1 as the sole evaluation metric.

E. LLM-as-a-judge evaluation

The F1 score measures how closely a prediction matches the oracle, but as Section III.D showed, a name can be perfectly readable while scoring low on F1 simply because it is more descriptive than the oracle. An alternative is to ask an LLM to assess readability directly, the way a human reviewer would.

In the *LLM-as-a-Judge* paradigm [14], an LLM is given a piece of code and a rubric — a list of concrete criteria — and asked to score each criterion. For example, one criterion might be: "Do the variable names reflect the values they hold?" The judge reads the test, checks it against each criterion, and produces a score. Because the judge reasons about meaning rather than counting matching tokens, it can recognize that `testYearEndReturnsDecember31` and `yearEnd` are both valid names for the same identifier.

To make scores reliable, two design choices matter. First, criteria must be specific and independently assessable — "is this readable?" produces inconsistent scores, whereas "does the method name describe the scenario being tested?" does not [15]. Second, using an ensemble of multiple judge models and averaging their scores reduces the risk that one model's quirks skew the result [16]. LLM judges configured this way have been shown to agree with human reviewers in over 80% of cases [14].

F. Large language models for code

LLMs are transformer-based models — a type of neural network that processes entire sequences of tokens at once, learning which parts of the code are most relevant to each other — pre-trained on large corpora of code and natural language [17]. This pre-training gives them an implicit understanding of naming conventions and the relationship between code structure and semantic intent — something that earlier task-specific models had to be explicitly engineered to capture [5], [18]. Code-specialized variants such as Qwen2.5-Coder [19] push this further by training predominantly on source code across many programming languages, achieving strong performance on a range of code understanding and generation benchmarks. LLMs have been shown to perform rename refactorings successfully, suggesting they have internalized sufficient naming knowledge from pre-training alone [20]. A key practical finding is that LLM performance on naming tasks improves significantly when the full function body is provided as context, rather than the identifier in isolation [21] — which motivates the design of our renaming pipeline.

G. Fine-tuning

Pre-trained LLMs can be adapted to a specific task by continuing training on a small, task-relevant dataset, a process known as fine-tuning [22], [23]. The model retains the broad knowledge acquired during pre-training while adjusting its

behavior to match the patterns in the new data. In our setting, fine-tuning means training on pairs of obfuscated tests and their original human-written identifiers, so the model learns to recover meaningful names from code structure alone. A useful property of fine-tuning is that it does not require training from scratch — even a small number of examples on a task-relevant dataset can meaningfully shift a model's output style and accuracy [22].

However, it is not always clear how much fine-tuning is needed to achieve the best result on a specific task. Performance gains from fine-tuning do not increase linearly with training — improvements tend to be largest in early stages and taper off as training continues [23]. For identifier renaming in test code specifically, this has not yet been studied. To investigate this, we train the model on the full training set and save a snapshot of the model's state — all its learned parameters at that point in training — each time it reaches 15%, 25%, 50%, 75%, and 100% of the total training steps. These snapshots are called *checkpoints*, and each one represents a version of the model trained on a progressively larger portion of the data seen so far.

H. Dataset

The dataset used in this study is Method2Test [24], one of the largest publicly available collections of Java unit tests paired with their corresponding production methods. It contains 780,944 test–method pairs drawn from open-source GitHub repositories, split into training, validation, and test sets, and has been widely used in unit testing research [25], [26]. For this study, only the test cases themselves are used — the production methods are not provided to any of the models, since the renaming pipeline operates on test bodies in isolation. After filtering out malformed entries — tests with unclosed braces, truncated functions, or inconsistent string literals — the dataset contains 414,976 training samples, 50,778 validation samples, and 52,472 test samples.

IV. Related works

The example in Section II raises a question that prior work has approached from three directions: how to rename identifiers automatically, how to improve the readability of generated tests using LLMs, and how to measure whether a renaming actually worked. Together, these strands tell us what is already known — and reveal what is not.

A. What we know

In this section we look at different prior works and look at what they found.

a) Identifier Renaming Without LLMs

The earliest attempts at automatic identifier renaming treated code as a sequence of tokens and applied statistical language models to predict names from context. Mastropaolo et al. compared an n-gram model, a T5 model, and a BERT-based model (CugLM) for variable renaming in production Java code [27]. Even the best model — CugLM at 63.5% complete match — struggled whenever multiple valid names existed for the same variable, which is precisely the situation we saw in Table I.

A second line of work recognized that token sequences alone miss important structural information and instead built graph representations of code. Allamanis et al. applied Gated Graph Neural Networks (GGNNs) to Abstract Syntax Trees augmented with dataflow edges, reaching 32.9% sub-token-level precision on production Java — roughly one in three sub-tokens predicted correctly [5]. De Keersmaeker adapted both GGNN and RefBERT specifically for automatically generated tests, the same setting as our study [6]. RefBERT outperformed GGNN but both models tended to produce names that were too generic, missing the test scenario entirely. The best result — RefBERT at F1 = 0.365 — is the direct non-LLM baseline our study compares against.

b) LLM-Based Test Improvement

The first work to directly target readability of automatically generated tests was DeepTC-Enhancer [28], which combined template-based test summarization with deep learning to suggest variable names. A user study found that 48% of suggestions fully conveyed the intended meaning — a promising start, but leaving more than half of cases incomplete. Around the same time, Daka et al. showed that even simple rule-based method naming from coverage criteria can reach 53% developer agreement [3], establishing that the naming problem is tractable even without deep learning.

The most closely related work to ours is Biagiola et al., who used LLMs to rename identifiers and test method names in EvoSuite-generated tests [7]. They evaluated nine models spanning proprietary APIs such as GPT-4 and Gemini, and open-source models accessed through Amazon Bedrock such as Llama 3, Claude 3, and Mistral — all used without any task-specific training, by providing the LLM with both the test and the production method it exercises as context. A study with ten professional developers found LLM-renamed tests as readable as developer-written ones, regardless of which model was used — the strongest evidence yet that LLMs can close the readability gap left by automated generation tools.

c) Readability Evaluation of Test Code

Early work on measuring code readability focused on structural features such as line length, nesting depth, and comment density. Scalabrino et al. showed that these metrics are insufficient to capture test readability, because automatically generated tests are doubly penalized: they lack comments entirely and have poor identifiers, meaning structural metrics alone cannot distinguish a well-named test from a poorly-named one [4].

Reference-based metrics such as BLEU — which count how many words in a prediction appear in the expected answer — fare no better for identifier evaluation. Evtikhiev et al. showed that BLEU scores do not correlate with human judgement of code quality [29], and Mastropaolo et al. found the same problem we saw in Table I: a correct rename scores near zero simply because multiple valid names exist for the same variable [27].

Winkler et al.'s systematic mapping study of test readability research confirmed that no single metric captures all dimensions of test readability, and that human studies remain the

most reliable evaluation method — but do not scale to large test suites [13]. Takebayashi et al. added a further dimension: expressive assertions and meaningful assertion messages are an often overlooked readability factor in test code [30].

Taken together, this line of work establishes that measuring readability is genuinely hard — structural metrics miss identifier quality, reference-based metrics penalize correct but unexpected names, and human studies cannot scale. This motivates the use of LLM-based evaluation as an alternative, which we discuss in Section III.D and apply in our comparative study.

B. What we do not yet know

The three strands of prior work paint a clear picture of progress: non-LLM approaches can rename identifiers automatically but top out at F1 = 0.365, LLMs can produce tests as readable as developer-written ones, and human studies are the most reliable evaluation method but do not scale. But three concrete questions remain unanswered.

Can LLMs actually beat the non-LLM baseline on a controlled benchmark? Biagiola et al. demonstrate that LLM-renamed tests are readable, but they do not compare against GGNN or RefBERT on the same data. It is therefore unknown whether the move to LLMs represents a meaningful improvement in naming accuracy over the best non-LLM approach, or simply a change in style.

How much task-specific training does it take to make a difference? No prior work has studied how naming quality evolves as a model trains on renamed test code — whether the gains appear immediately or only after extensive training. This matters in practice: a model trained for a fraction of the full schedule is far cheaper to produce, and if most of the benefit appears early, full training may not be worth the cost.

Does better naming accuracy mean more readable tests? The example in Section II already hints that these two properties can pull in opposite directions: `testYearEndReturnsDecember31` scores 0.50 against the oracle `yearEnd` yet is arguably the clearer name. Whether this tension is systematic across a full benchmark — and whether a model that scores higher on F1 is actually more useful to a developer — has not been studied.

These three gaps motivate the research questions we address in Section V.

V. Approach

This section describes the comparative study design: the research questions it addresses, the dataset, the renaming pipeline used to produce renamed tests, the models compared, and the evaluation metrics.

A. Research Questions

The three gaps identified in Section IV motivate the following research questions.

RQ1: Do LLM-based approaches outperform non-LLM approaches at renaming identifiers in automatically generated tests? Prior work established RefBERT as the best non-LLM approach, achieving F1 = 0.365 on generated test code [6]. This question asks whether LLMs — both with and without task-specific training — represent a meaningful step forward on the same benchmark.

RQ2: How much task-specific training is needed before gains appear? We fine-tune a model at five checkpoints (15%, 25%, 50%, 75%, and 100% of total training steps) and measure naming quality at each. This question asks whether the benefit of fine-tuning appears early or only after extensive training — a practically important question given the computational cost of training.

RQ3: Does a model that scores higher on naming accuracy also produce more readable tests? F1 measures how closely a predicted name matches the oracle, but as Section II showed, a more descriptive name can score lower than a concise one. This question asks whether naming accuracy and holistic readability move together across approaches, or whether optimizing for one comes at the cost of the other. Then the choice of metric changes which approach a practitioner would adopt, and both dimensions are necessary to get a complete picture.

B. Dataset setup

The dataset used in this study is Method2Test [24], described in Section III. After filtering out malformed entries, it contains 414,976 training samples, 50,778 validation samples, and 52,472 test samples. The full filtered dataset is used for fine-tuning Qwen2.5-Coder, preserving the original 80/10/10 split.

For evaluation, we randomly sample 100 test methods from the held-out test split using a fixed seed, ensuring reproducibility. The sample size was determined through a prospective power analysis on a pilot run of 30 test methods, following the statistical recommendations of Arcuri and Briand [31]. 100 samples is statistically adequate for all primary comparisons in this study — full details are provided in Appendix E.

Each test method is obfuscated as described in Section III: function names are replaced with `func_n` and variable names with `var_n`. Each obfuscated test is paired with its original human-written version, which serves as the oracle for F1 evaluation. Examples of the obfuscated format and the JSONL structure used for fine-tuning are shown in Listing 6 and Listing 7.

C. The renaming pipeline

The renaming pipeline takes an obfuscated test as input and returns the same test with all identifiers replaced by meaningful names. Fig. 1 gives an overview of the full process.

The pipeline works in three steps. First, the identifiers are extracted from the obfuscated test — the method name and all local variable names — and the test body is prepared as input. Second, the model receives a prompt containing the test body and is asked to return a JSON object mapping each old placeholder name to a new meaningful one. Using a structured JSON format makes the output predictable and easy to validate automatically [21]. Third, the mapping is validated: if the response is malformed, incomplete, or contains names that do not follow Java naming conventions, the model is asked to

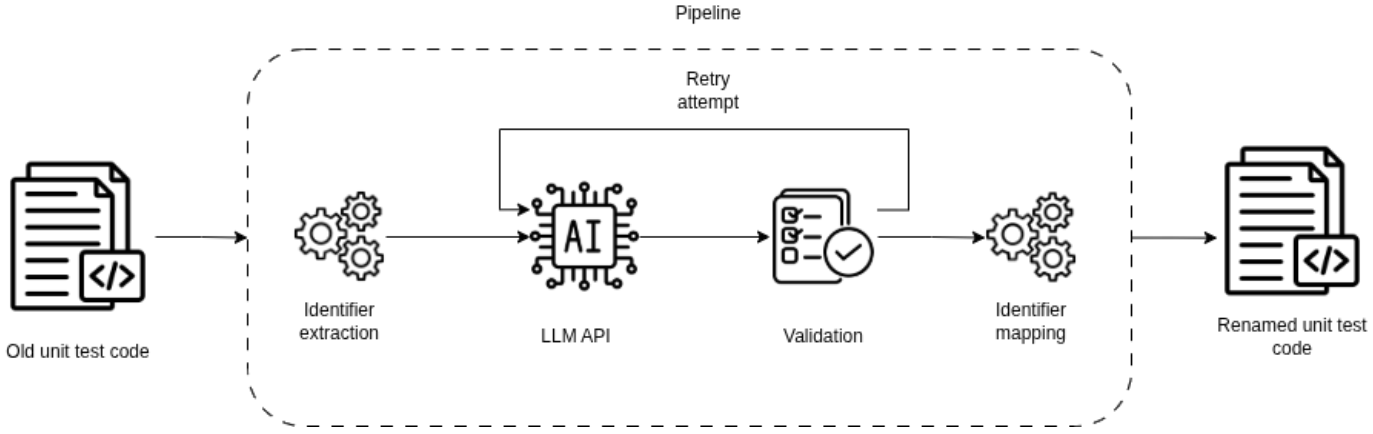

Fig. 1. Overview of the renaming pipeline. Identifiers are extracted from the obfuscated test, passed to the model, and the response is validated before the mapping is applied. Invalid responses trigger an automatic retry, up to three attempts.

try again with explicit feedback about what went wrong, up to three attempts [32]. Once a valid mapping is obtained, the placeholders in the original test are replaced and the renamed test is written out.

The same prompt format is used for all models — both fine-tuned and non-fine-tuned — so that differences in output reflect the models themselves rather than differences in how they were instructed. The full prompt templates are provided in Appendix A.

D. Models compared

We compare seven systems across two categories: non-LLM baselines and LLM-based approaches.

Non-LLM baselines. We include GGNN [5] and RefBERT [6] as the two non-LLM approaches on the same task, re-evaluated on our 100-sample test subset. These serve as the reference point for RQ1.

LLM-based approaches. We evaluate five LLM-based systems. GPT-4.1 mini and GPT-5 mini are accessed through the OpenAI API without any task-specific training. Qwen2.5-Coder-7B-Instruct is a locally deployed open-source model evaluated in two settings: first as a base model without any task-specific training, to isolate the effect of fine-tuning; then fine-tuned on the Method2Test training set at five checkpoints — 15%, 25%, 50%, 75%, and 100% of total training steps — to study how naming quality evolves during training (RQ2).

Table II summarizes all compared systems.

TABLE II
SYSTEMS COMPARED IN THIS STUDY.

System	Type	Training
GGNN	Non-LLM	De Keersmaeker et al. dataset
RefBERT	Non-LLM	De Keersmaeker et al. dataset
GPT-4.1 mini	LLM (API)	None
GPT-5 mini	LLM (API)	None
Qwen2.5-Coder (base)	LLM (local)	None
Qwen2.5-Coder (×5 checkpoints)	LLM (local)	Fine-tuned on Method2Test

E. Evaluation

We evaluate each system on two dimensions: naming accuracy and readability. Using two complementary metrics is central to this study — as Section II showed, a system can score well on one while performing poorly on the other.

a) Naming Accuracy: Subtoken-level F1

Naming accuracy is measured using the sub-token-level F1 score, described in Section III.D. For each test in the evaluation set, we compute F1 between the predicted identifier names and the oracle names, then average across all identifiers and all tests. The non-LLM baselines (GGNN and RefBERT) are re-evaluated on our 100-sample subset using the same metric to ensure a fair comparison.

b) Readability: CodeReader

Readability is measured using CodeReader, an open-source LLM-based grading tool developed as part of this study. CodeReader presents each renamed test to an ensemble of LLM judges, which score it against a rubric of six readability criteria — covering aspects such as whether the method name describes the scenario being tested, whether variable names reflect the values they hold, and whether the test avoids unexplained magic numbers. Using an ensemble of judges rather than a single model reduces the risk that one model's quirks skew the result [16]. The full rubric and judge configuration are provided in Appendix C.

Both metrics are computed on the same 100-sample evaluation subset, allowing a direct comparison across all systems on both dimensions.

VI. Results

We report results for each research question in turn, using the same 100-sample evaluation subset across all systems to ensure a fair comparison.

A. RQ1: Do LLM-based approaches outperform non-LLM approaches?

Table III reports the naming accuracy and readability scores for all compared systems.

TABLE III
Naming accuracy (F1) and readability scores for all compared systems on the 100-sample evaluation subset.

System	Precision	Recall	F1	Readability
GGNN	0.270	0.230	0.241	63.2
RefBERT	0.320	0.306	0.308	55.4
Qwen (untuned)	0.714	0.817	0.760	70.0
Qwen (15%)	0.848	0.825	0.834	65.1
Qwen (25%)	0.853	0.831	0.839	64.8
Qwen (50%)	0.849	0.828	0.836	65.8
Qwen (75%)	0.853	0.824	0.836	63.6
Qwen (100%)	0.849	0.826	0.835	65.6
GPT-4.1 mini	0.730	0.835	0.776	69.4
GPT-5 mini	0.699	0.831	0.756	70.2

The two non-LLM baselines tell different stories. GGNN achieves F1 = 0.241 on our evaluation subset — lower than the F1 = 0.296 reported by De Keersmaeker [6]. Two factors explain this discrepancy. First, De Keersmaeker evaluated on the full Method2Test dataset while we evaluate on a 100-sample subset, meaning our results are more sensitive to individual cases. Second, GGNN requires each test file to be converted to a specific intermediate format as part of its graph construction process — a conversion that does not always succeed, and failed conversions result in not performing any renaming, directly lowering the F1 score (see Appendix J for details). RefBERT fares considerably better at F1 = 0.308, close to De Keersmaeker's reported F1 = 0.365, confirming it as the meaningful non-LLM baseline for this task.

Every LLM-based system substantially outperforms both baselines. The untuned Qwen model achieves F1 = 0.760 — more than doubling RefBERT — without any task-specific training, purely on the basis of general code knowledge acquired during pre-training. GPT-4.1 mini and GPT-5 mini achieve F1 = 0.776 and F1 = 0.756 respectively, comparable to the untuned Qwen model. The fine-tuned Qwen checkpoints push further, reaching F1 between 0.834 and 0.839.

Returning to the `yearEnd` example from Section II: RefBERT correctly renamed the identifiers for this test, producing `date` and `calendar` matching the oracle exactly. However, this does not generalise consistently across the full evaluation set, where RefBERT achieves F1 = 0.308 — less than half the score of any LLM-based approach.

Answer to RQ1: LLM-based approaches substantially outperform non-LLM baselines on naming accuracy, more than doubling the F1 score of the best non-LLM approach regardless of whether the LLM was fine-tuned or not.

Unlike GGNN that depend on a file-specific format or require code to be compilable, LLM-based approaches operate directly on source code as text. This makes them inherently more flexible, as they can process code tasks independently of the compilation state or the constraints of a particular file format.

B. RQ2: How much fine-tuning is needed

Table III already shows the answer clearly: the largest gain from fine-tuning appears immediately. The untuned Qwen model achieves F1 = 0.760, and after just 15% of total training steps this jumps to F1 = 0.834 — a gain of 0.074 in a single step. From 15% onward, F1 remains essentially flat, ranging between 0.834 and 0.839 across all remaining checkpoints.

This pattern of rapid early gain followed by saturation suggests that the model quickly learns the structural patterns of the obfuscated dataset — how placeholder names relate to the surrounding code structure — after which additional training steps yield diminishing returns [23].

A secondary finding concerns schema compliance. The untuned model produced 7 invalid responses across 100 files, where it returned additional identifiers beyond those listed in the prompt. From the 15% checkpoint onward, every fine-tuned model produced zero schema errors, indicating that fine-tuning on the structured JSON output format quickly improves instruction-following reliability — a practical benefit beyond the F1 gain itself.

Answer to RQ2: Most of the benefit of fine-tuning on naming accuracy appears within the first 15% of training steps and saturates rapidly beyond that point. A model trained to full convergence offers no meaningful F1 advantage over one trained for a fraction of the schedule.

C. RQ3: Does accuracy translate to readability

Looking at the readability column of Table III, a clear pattern emerges that runs counter to what F1 scores alone would suggest.

The untuned Qwen model achieves the highest readability score of all Qwen variants at 70.0, and this drops consistently as fine-tuning increases — falling to 65.1 at 15%, 64.8 at 25%, and remaining lower through all checkpoints. The fine-tuned model that scores highest on F1 (0.839 at 25%) scores among the lowest on readability (64.8). GPT-4.1 mini and GPT-5 mini, despite F1 scores comparable to the untuned Qwen model, achieve readability scores of 69.4 and 70.2 respectively — on par with or slightly above the untuned model.

The `TestClass280` test illustrates concretely why this divergence occurs. The oracle names the two variables simply `a` and `b` — minimal names that match a terse developer style.

```java
@Test public void compare() {
    long a = 5;
    long b = 4;
    assertTrue(a > b);
    assertFalse(a < b);
}
```

Listing 3. Oracle version of `TestClass280`.

The fully fine-tuned Qwen model learned to reproduce exactly this terse oracle style, producing `a`, `b`, and `testLongLessThan` — earning F1 = 0.93, the highest of any system on this test.

```java
@Test public void testLongLessThan() {
    long a = 5;
    long b = 4;
    assertTrue(a > b);
    assertFalse(a < b);
}
```

Listing 4. Qwen 100% output for `TestClass280` (F1 = 0.93, readability = 57.7).

But `a` and `b` tell a developer nothing about what the variables represent. The untuned model, never trained on the task, produces `firstNumber` and `secondNumber` — names that immediately convey meaning — and scores readability = 70.0 despite its F1 of only 0.32.

```java
// The full function name =
 // testShouldReturnFalseWhen
 // FirstNumberIsLessThanSecond
@Test public void testShouldReturnFalseWhenEtc()
{
    long firstNumber = 5;
    long secondNumber = 4;
    assertTrue(firstNumber > secondNumber);
    assertFalse(firstNumber < secondNumber);
}
```

Listing 5. Qwen untuned output for `TestClass280` (F1 = 0.32, readability = 70.0).

GPT-4.1 mini and GPT-5 mini follow the same pattern, producing descriptive names like `largerValue`, `smallerValue`, `actualValue`, and `expectedValue` — all scoring readability above 69 despite F1 scores below 0.40. Notably, even the oracle illustrates the limitation of reference-based evaluation: `a` and `b` are valid Java identifiers, but a developer reading this test cold learns no more from them than from the original placeholders `var_1` and `var_2`.

This divergence is systematic across the full evaluation set, not just this one test. Fine-tuning causes the model to converge toward the naming conventions present in the Method2Test dataset, which contains known identifier quality issues [11]. A model that learns to reproduce those conventions scores higher on F1 while producing names that the judge ensemble rates as less readable — because conciseness and F1 accuracy are not the same property as readability.

Answer to RQ3: Higher naming accuracy does not translate into more readable tests. Fine-tuning substantially improves F1 while simultaneously reducing readability scores, and the approaches that score highest on readability are those that were never trained on the task. Evaluating identifier renaming tools on F1 alone gives a misleading picture of which approach is most useful to a developer.

VII. Discussion

This section interprets the results of the three research questions, discusses what they mean for future work on identifier renaming, and considers the threats to the validity of the study.

A. Summary of findings

The three research questions together tell a coherent story about LLM-based identifier renaming.

RQ1 establishes that LLMs represent a substantial step forward over non-LLM approaches. Even without any task-specific training, an LLM more than doubles the naming accuracy of RefBERT — the best non-LLM baseline — purely on the basis of general code knowledge acquired during pre-training. This suggests that the broad understanding of naming conventions that LLMs develop from large code corpora transfers directly to the renaming task, without requiring task-specific adaptation.

RQ2 shows that fine-tuning provides an immediate and large accuracy gain, but most of that gain appears within the first 15% of training steps and saturates rapidly beyond that point. This is a practically important finding: a model trained for a fraction of the full schedule achieves nearly the same F1 as one trained to full convergence, at a fraction of the computational cost.

RQ3 reveals the most consequential finding: the accuracy gain from fine-tuning comes at the cost of readability. Fine-tuning causes the model to converge toward the naming conventions present in the training data — conventions that are not always of high quality [11] — producing names that score well on F1 but that a developer finds less informative than the more descriptive names produced by a model that was never trained on the task. The `TestClass280` example makes this concrete: `a` and `b` match the oracle and earn F1 = 0.93, but `firstNumber` and `secondNumber` tell a developer immediately what the test is comparing.

Taken together, these findings suggest that F1 and readability are measuring fundamentally different properties of a renamed test, and that a tool optimizing for one will not necessarily perform well on the other. Neither metric alone is a sufficient ground truth for evaluating identifier renaming tools — both should be reported together.

B. Threats to validity

This study follows the validity categorization of Wohlin et al. [33], considering construct, internal, external, and conclusion validity.

a) Construct Validity

The primary construct validity threat concerns the LLM readability score. While the scoring criteria are grounded in established identifier quality research [4], [11], the score is

produced by an ensemble of LLM judges rather than human evaluators, and may not fully capture what developers consider readable in practice. The absence of a human study means we cannot verify that judge scores correlate with actual developer preference on our specific dataset.

Three interrelated limitations compound this threat. First, the judge models are non-deterministic — scoring the same test twice may produce slightly different scores. The reported readability scores reflect a single evaluation run per file, and inter-run variance was not formally measured. Using an ensemble of three diverse judge models partially mitigates this by producing a more stable aggregated score than any single model [16], but residual non-determinism remains inherent to any LLM-based evaluation. Second, the ensemble may have an implicit preference for longer, more descriptive names — the style produced by untuned models — which could partially explain why untuned models score higher on readability. Third, different judge model selections may produce different relative rankings. Without a human study to validate CodeReader scores against developer preference, these biases cannot be fully ruled out.

b) Internal Validity

The Method2Test dataset contains known identifier quality issues [11] — a non-trivial fraction of human-written test identifiers are themselves of poor quality. This means the oracle labels used for F1 computation are not a reliable upper bound on naming quality, and a model that converges toward those labels may score well on F1 while producing names that are no more readable than the obfuscated originals. The `TestClass280` oracle — which uses `a` and `b` — is a direct example of this.

c) External Validity

The evaluation is limited to Java test methods sampled from Method2Test. Results may not generalize to other programming languages, other test generation tools, or test methods from different domains.

A further threat concerns the use of obfuscated human-written tests rather than genuinely auto-generated tests. While obfuscation simulates the surface-level naming patterns of tools like EvoSuite and Randoop, truly auto-generated tests tend to be shorter, contain fewer variables, and lack the semantic coherence of human-written code [8]. A renaming model trained and evaluated on obfuscated human-written tests may encounter structurally simpler inputs at deployment time, potentially affecting the F1 and readability scores reported here. Evaluating the pipeline on real EvoSuite and Randoop output is a priority for future work.

Additionally, all fine-tuning experiments use a 7B parameter model due to hardware constraints, and results may not generalize to larger models. However, the two central findings are expected to hold more broadly: the benefit of fine-tuning on token-level metrics is well established across model scales [34], [35], and the divergence between F1 and readability is a property of the metrics themselves rather than of any specific model [27], [29].

d) Conclusion Validity

As established in Appendix E, 100 samples is statistically adequate for all primary comparisons in this study. The one exception is the comparison between renamed and oracle output at some checkpoints, where the effect is small and conclusions should be treated as indicative rather than definitive.

C. Future work

This study opens several directions for future research.

Validation with a human study. The most direct extension is to validate CodeReader scores against developer preference on a sample of renamed tests. Such a study would confirm whether the readability divergence observed in RQ3 reflects genuine developer experience, and would establish how well LLM judge scores correlate with human judgement for this specific task.

Evaluation on genuinely auto-generated tests. Our pipeline was trained and evaluated on obfuscated human-written tests. Evaluating it on real EvoSuite or Randoop output would reveal whether the findings hold for the structurally simpler tests that these tools produce in practice, and whether the renaming pipeline degrades on inputs that differ from its training distribution.

Joint optimisation of accuracy and readability. The central finding of this study is that F1 and readability can pull in opposite directions. Future work could explore training objectives that optimise for both simultaneously — for example, using CodeReader scores as a reward signal in a reinforcement learning from human feedback (RLHF) style setup, incorporating readability criteria directly into the fine-tuning loss or making an dataset that is filtered based on some readability criteria.

VIII. Conclusion

We started with a simple question: given a test where a developer sees `func_1`, `var_1`, and `var_2` instead of meaningful names, which approach produces the best result?

For the `yearEnd` test from Section II, every LLM-based approach we compared produced names a developer can immediately understand — `actualDate`, `testShouldReturnYearEndMonthAndDay`, `actualCalendar`. While RefBERT — the best non-LLM approach — did produce correct variable names for this test, its overall F1 score of 0.308 across the full evaluation set shows that this does not generalize consistently. Every LLM-based approach achieves F1 above 0.75, more than doubling that result.

The more surprising finding comes from comparing LLM approaches against each other. The fine-tuned model, trained on thousands of renamed tests, learned to reproduce the concise oracle style — giving `var_2` the name `cal` and the method the name `testYearEnd`, close to the human-written reference. That makes it the most accurate system we compared, reaching F1 = 0.839. But the untuned model produces `actualCalendar` and `testShouldReturn YearEndMonthAndDay` — names that score lower on F1 yet tell a developer more at a glance about what the variable holds and what the test is asserting. The fine-tuned model

learned to match the reference; it did not learn to be more readable.

This is the central lesson: naming accuracy and readability are not the same thing, and the `yearEnd` example shows exactly why — `cal` and `actualCalendar` are both correct names for the same variable, but they are not equally informative. Evaluating only F1 would declare the fine-tuned model the winner; evaluating only readability would declare the untuned model the winner. Both metrics are needed to get the full picture.

To make this dual evaluation practical without requiring a human study for every new tool, we used CodeReader — an open-source tool that uses an ensemble of LLM judges to score test readability against an explicit rubric — as the readability metric in our comparison. We release it openly so that future work can evaluate identifier renaming tools on both dimensions without the cost of a human study.

References

[1] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, New York, NY, USA: ACM, 2011, pp. 416–419. doi: 10.1145/2025113.2025179.

[2] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-Directed Random Test Generation," in *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, Minneapolis, MN, USA: IEEE Computer Society, 2007, pp. 75–84.

[3] E. Daka, J. M. Rojas, and G. Fraser, "Generating Unit Tests with Descriptive Names or: Would You Name Your Children Thing1 and Thing2?," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, New York, NY, USA: ACM, 2017, pp. 57–67. doi: 10.1145/3092703.3092727.

[4] S. Scalabrino, M. Linares-Vásquez, R. Oliveto, and D. Poshyvanyk, "A Comprehensive Model for Code Readability," *Journal of Software: Evolution and Process*, vol. 30, no. 6, p. e1958, 2018, doi: 10.1002/smr.1958.

[5] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to Represent Programs with Graphs," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1711.00740

[6] A. De Keersmaeker, "Enhancing Test Code Understandability with Machine Learning-Based Identifier Naming," Master's Thesis, 2024.

[7] M. Biagiola, G. Ghislotti, and P. Tonella, "Improving the Readability of Automatically Generated Tests using Large Language Models," in *Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, Naples, Italy: IEEE, 2025.

[8] G. Fraser and A. Arcuri, "EvoSuite: On the Challenges of Test Case Generation in the Real World," in *Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2013, pp. 362–369. doi: 10.1109/ICST.2013.5954405.

[9] J. Wang, B. Suleiman, and M. J. Alibasa, "Automated Unit Test Case Generation: A Systematic Literature Review." [Online]. Available: https://arxiv.org/abs/2504.20357

[10] S. Butler, M. Wermelinger, Y. Yu, and H. Sharp, "Exploring the Influence of Identifier Names on Code Quality: An Empirical Study," in *Proceedings of the 14th European Conference on Software Maintenance and Reengineering (CSMR)*, Madrid, Spain: IEEE, 2010, pp. 156–165. doi: 10.1109/CSMR.2010.27.

[11] B. Lin, C. Nagy, G. Bavota, A. Marcus, and M. Lanza, "On the Quality of Identifiers in Test Code," in *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2019, pp. 204–215. doi: 10.1109/SCAM.2019.00031.

[12] B. Lin, S. Scalabrino, A. Mocci, R. Oliveto, G. Bavota, and M. Lanza, "Investigating the Use of Code Analysis and NLP to Promote a Consistent Usage of Identifiers," in *Proceedings of the 17th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM 2017)*, IEEE, 2017, pp. 81–90. doi: 10.1109/SCAM.2017.13.

[13] D. Winkler, P. Urbanke, and R. Ramler, "Investigating the Readability of Test Code: Combining Scientific and Practical Views," *Empirical Software Engineering*, vol. 29, no. 2, p. 53, 2024, doi: 10.1007/s10664-023-10390-z.

[14] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[15] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore: Association for Computational Linguistics, 2023, pp. 2511–2522. doi: 10.18653/v1/2023.emnlp-main.153.

[16] P. Verga *et al.*, "Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models." [Online]. Available: https://arxiv.org/abs/2404.18796

[17] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.

[18] A. Fan *et al.*, "Large Language Models for Software Engineering: Survey and Open Problems," in *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, IEEE, 2023, pp. 31–53. doi: 10.1109/ICSE-FoSE59343.2023.00008.

[19] B. Hui *et al.*, "Qwen2.5-Coder Technical Report." [Online]. Available: https://arxiv.org/abs/2409.12186

[20] J. Cordeiro, E. Noei, and Y. Zou, "An Empirical Study on the Code Refactoring Capability of Large Language Models," *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3801158.

[21] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, and P. Liu, "How Does Naming Affect Language Models on Code Analysis Tasks?," *Journal of Software Engineering and Applications*, vol. 17, no. 11, pp. 803–816, 2024, doi: 10.4236/jsea.2024.1711044.

[22] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, "Transfer Learning: A Friendly Introduction," *Journal of Big Data*, vol. 9, no. 1, p. 102, 2022, doi: 10.1186/s40537-022-00652-w.

[23] K. W. Church, Z. Chen, and Y. Ma, "Emerging Trends: A Gentle Introduction to Fine-Tuning," *Natural Language Engineering*, vol. 27, no. 6, pp. 763–778, 2021, doi: 10.1017/S1351324921000322.

[24] M. Tufano, S. K. Deng, N. Sundaresan, and A. Svyatkovskiy, "Methods2Test: A Dataset of Focal Methods Mapped to Test Cases," in *Proceedings of the 19th International Conference on Mining Software Repositories*, in MSR '22. ACM, May 2022.

[25] A. Storhaug and J. Li, "Parameter-Efficient Fine-Tuning of Large Language Models for Unit Test Generation: An Empirical Study." 2024.

[26] Y. Shang, Q. Zhang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "A Large-Scale Empirical Study on Fine-Tuning Large Language Models for Unit Testing," *Proceedings of the ACM on Software Engineering*, 2025, doi: 10.1145/3728951.

[27] A. Mastropaolo, E. Aghajani, L. Pascarella, and G. Bavota, "Automated Variable Renaming: Are We There Yet?," *Empirical Software Engineering*, vol. 28, no. 2, p. 45, 2023, doi: 10.1007/s10664-022-10274-8.

[28] D. Roy *et al.*, "DeepTC-Enhancer: Improving the Readability of Automatically Generated Tests," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, New York, NY, USA: ACM, 2020. doi: 10.1145/3324884.3416622.

[29] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the BLEU: How Should We Assess Quality of the Code Generation Models?," *Journal of Systems and Software*, vol. 203, p. 111741, 2023, doi: 10.1016/j.jss.2023.111741.

[30] T. Takebayashi, A. Peruma, M. W. Mkaouer, and C. D. Newman, "An Exploratory Study on the Usage and Readability of Messages within Assertion Methods of Test Cases." 2023.

[31] A. Arcuri and L. Briand, "A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, ACM, 2011, pp. 1–10. doi: 10.1145/1985793.1985795.

[32] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learn-

ing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[33] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.

[34] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9

[35] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[36] N. Jain *et al.*, "NEFTune: Noisy Embeddings Improve Instruction Finetuning," in *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://openreview.net/forum?id=0bMmZ3fkCk

[37] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[38] I. Olkin and J. W. Pratt, "Unbiased Estimation of Certain Correlation Coefficients," *Annals of Mathematical Statistics*, vol. 29, no. 1, pp. 201–211, 1958, doi: 10.1214/aoms/1177706717.

[39] R. A. Fisher, "Frequency Distribution of the Values of the Correlation Coefficient in Samples from an Indefinitely Large Population," *Biometrika*, vol. 10, no. 4, pp. 507–521, 1915, doi: 10.2307/2331838.

[40] Vast.ai, "Vast.ai: GPU Cloud Computing Platform." 2024.

[41] Q. Zhu and others, "DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence." 2024.

[42] M. Abdin and others, "Phi-4 Technical Report." [Online]. Available: https://arxiv.org/abs/2412.08905

[43] M. Abdin and others, "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone." [Online]. Available: https://arxiv.org/abs/2404.14219

[44] Qwen Team, "Qwen3.5: A Family of Open-Source Multimodal Models." 2025.

Appendix A: Prompt Templates

The renaming pipeline uses three prompt templates. Prompt 1 is sent as the system instruction to establish the model's role and output constraints. Prompt 2 is the user-facing prompt providing the obfuscated test method and identifier list. Prompt 3 is the retry prompt, invoked when a response fails schema validation following the Reflexion framework [32].

Appendix A.1: System prompt

```
You are a Java test code refactoring expert.
You will be given:
- A Java test method (wrapped in a dummy
class), and
- A list of identifiers to rename, each tagged
with its kind: (method) or (variable).

Your job is to propose meaningful names for ALL
of the listed identifiers.

Naming conventions to follow:
- Methods (func_*): use camelCase starting with
'test',
  e.g. testShouldReturnNullWhenInputIsEmpty.
- Variables used in assertions (var_*): prefer
'expected' / 'actual' where applicable.
- Other variables (var_*): use concise
camelCase nouns that describe the value,
  e.g. userCount, resultList.

Rules:
- You MUST include every identifier from the
list as a key - no omissions.
- Use ONLY the listed identifiers as keys - no
extras.
- You MUST ONLY respond with a valid JSON
object mapping originalName -> newName.
- Do NOT output code, comments, markdown, or
any text outside the JSON object.
```

Prompt 1. System Instruction Prompt

Appendix A.2: User prompt

```
Here is the obfuscated Java test method wrapped
in a dummy class:

```java
{test_case}
```

Here are ALL the identifiers that must be
renamed:
{identifiers}

Rename ALL of the identifiers above. You MUST
include every one of them as a key.
Return a single JSON object mapping
originalName -> newName.

Example (for a method + two variables):
{"func_1": "testShould...",
 "var_1": "actualResult",
 "var_2": "expectedSize"}

Rules:
- ALL listed identifiers must appear as keys in
your response.
- Use ONLY the listed identifiers as keys - do
NOT add or remove any.
- Do NOT output anything except the JSON object
(no backticks, no text).
```

Prompt 2. User Prompt Template

Appendix A.3: Retry prompt

```
Here is the original obfuscated Java test
method wrapped in a dummy class:

```java
{test_case}
```

Here are ALL the identifiers that must be
renamed:
{identifiers}

Your previous response was rejected for this
reason:
{error_reason}

Your previous (rejected) response was:
{failed_response}

Please try again. Make sure your new response:
- Includes EVERY identifier from the list above
as a key.
- Uses ONLY the listed identifiers as keys.
- Is a valid JSON object only - no backticks,
no explanation.
```

Prompt 3. Retry Prompt Template

Appendix B: Training Configuration

The model is trained as a causal language model on sequences composed of the prompt followed by the completion, where the completion is a single JSON object mapping all obfuscated identifier names in the method to their original names simultaneously.

Prompt tokens are masked in the labels tensor, meaning the cross-entropy loss is computed exclusively over the comple-

tion tokens, the predicted identifier names, rather than over the prompt itself. This is a standard fine-tuning practice that prevents the model from wasting capacity learning to reproduce its own input, and instead focuses all gradient updates on the renaming output, ensuring the model learns to map obfuscated identifiers to meaningful names.

Next, adapters are injected into all seven projection matrices of each Transformer block (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), which has been shown to improve downstream performance over targeting attention projections alone [34]. The adapter parameterization follows LoRA [34]: for a weight matrix $W \in \mathbb{R}^{\{d\times k\}}$, the update is decomposed as $\Delta W = BA$ where $B \in \mathbb{R}^{\{d\times r\}}$ and $A \in \mathbb{R}^{\{r\times k\}}$ with rank $r \ll \min(d, k)$. Optimization uses paged AdamW in 8-bit precision with a cosine learning rate schedule and a 3% linear warmup.

Additionally, we apply NEFTune [36]. This is to reduce overfitting on the fine-tuning dataset. NEFTune adds uniform random noise to the embedding vectors during each forward pass. It acts as an implicit regularizer: it has been shown to increase training loss while simultaneously lowering test loss on held-out instructions, directly counteracting memorization of training examples [36]. This property is particularly relevant here, since our fine-tuning dataset consists of obfuscated test methods drawn from a single distribution, making memorization a realistic risk.

Appendix C: Grading Tool Configuration

Listing 1, Listing 2 and Listing 3 show the complete configuration file used for CodeReader in this study. The configuration is split into two parts: the scoring and rules definition (Listing 1), and the model ensemble definition (Listing 2).

The scoring section defines a 0–100 scale with identifier naming as the primary focus. The four weighted criteria reflect their relative importance to the renaming task: identifier naming clarity carries the dominant weight (0.75) as it directly measures the quality of the renaming output; behavioral specificity of the test method name (0.15) captures whether the test scenario is communicated through the method name following the established JUnit convention [3]; and assertion quality (0.10) provides a secondary signal on the expressiveness of the test body [30]. Test independence is assigned a weight of zero since the tool evaluates each test method in isolation, making cross-test dependencies unobservable and the criterion inapplicable in this setting.

The eight evaluation rules are derived from established research on identifier quality and test readability, and operationalise the scoring criteria as concrete, independently assessable instructions following the G-Eval recommendation [15], which shows that explicit fine-grained criteria produce more stable and calibrated scores than vague numeric scales.

Rules 1 and 2 — Clarity and penalization of auto-generated names. Butler et al. demonstrated empirically that poor identifier naming directly degrades a developer's ability to understand and maintain code [10]. Lin et al. further showed that only 61.6% of test identifiers are rated acceptable compared to 92.5% in production code [11], confirming that naming quality in test code is a persistent problem. These findings motivate explicitly penalizing names that appear still auto-generated (e.g., `var0`, `obj1`), as these represent a complete failure of the renaming step.

Rule 3 — Cross-identifier consistency within a single test. A test method where naming quality is uneven across its own variables is harder to understand than one where all identifiers follow a coherent strategy, even if individual names score well in isolation. Scalabrino et al. identified intra-method naming consistency as a readability factor [4], and Lin et al. found that inconsistent naming within test code is a persistent quality problem [11]. This is distinct from file-level consistency: the rule penalizes tests where, for example, one variable is named `expectedUserAge` while a semantically related variable in the same method is named `result2`, regardless of how other test methods in the file are named.

Rule 4 — Alignment with actual test logic. A name that does not match what a variable actually holds is actively misleading and degrades comprehension more than a generic name would [10]. Scalabrino et al. include semantic alignment between identifiers and their role in the code as a key readability dimension [4].

Rule 5 — Test method naming and behavioral specificity. Daka et al. established that test method names following the pattern `methodName_state UnderTest_expectedBehavior` significantly improve developer understanding of the tested behaviour, and that names like `test42` serve neither documentation nor comprehension purposes [3]. Winkler et al. confirmed that method naming is among the primary readability factors in test code [13].

Rule 6 — Assertion quality and magic numbers. Takebayashi et al. found that expressive assertions are an important but frequently overlooked readability factor in test code, and that hard-coded literals without named constants make the intent of assertions non-obvious [30].

Rule 7 — Test independence. Although test independence is weighted zero in our configuration because the tool evaluates each test method in isolation, the rule is retained in the rubric to maintain completeness and to allow future configurations that evaluate test files as a whole.

Rule 8 — Ignore class name. The enclosing class name is excluded from scoring because it falls outside the scope of the renaming task, which targets only local variable and test method names. Penalizing a generic class name would introduce noise into the score that does not reflect the quality of the renaming output.

```yaml
language: java

settings:
  timeout_seconds: 320
  health_timeout_seconds: 900
  max_concurrency: 1
  fail_on_no_active_models: true
  log_path: out/temp/readability_results.txt
  debug_jsonl_path: responses.jsonl

scoring:
  scale: 0-100
  focus: identifier_naming
  weights:
    identifier_naming: 0.75
    assertion_quality: 0.10
    test_independence: 0
    behavioral_specificity: 0.15

tags:
  - testing
  - unit tests
  - identifiers
  - function naming
  - variable naming
  - assertion clarity
  - test independence
```

Listing 1. Codereader Configuration File (`codereader.yml`) — Part 1

```yaml
rules:
  - Clarity of variable and method names assess whether identifier names clearly
    convey intent without needing to read the full test body. Penalise generic
     names like var1, obj, result0, func_1 or single letters unless used as counters.
    - Renamed vs. generic baseline explicitly penalise identifiers that appear to
     still be auto-generated (e.g. var0, obj1, int0, string1, list0). These
     represent failures of the renaming step and must receive the maximum penalty
    on the identifier dimension.
   - Identifier consistency across the test all identifiers within a single test
     method should follow a coherent naming strategy. If one variable is named
     expectedUserAge, a related variable should not be named result2. Penalise
     tests where naming quality is uneven across variables.
  - Alignment with actual test logic identifier names must match what the
     variable actually holds or what the method actually does. A variable named
    expectedResult holding an exception object is misleading and must be flagged.
  - Test method naming and behavioral specificity the test method name should
                    follow the pattern methodName_stateUnderTest_expectedBehavior or a similar
     convention. The name and body together must make the tested behaviour and
    expected outcome obvious without reading the production code.
  - Assertion quality and magic numbers assertions must be expressive and clearly
       state what is being verified. Prefer assertEquals, assertNotNull, assertThrows
     over assertTrue(a == b). Penalise absent assertion messages when the failure
     reason would be non-obvious. Flag hardcoded numeric or string literals with no
     named constant or comment explaining their meaning - well-named expected value
     variables (e.g. int expectedAge = 42) are strongly preferred.
   - Test independence the test must be self-contained and not rely on shared
     mutable state or execution order. Penalise tests that depend on external state
    or side effects from other tests.
  - Ignore class name do not penalise or reward based on the outer class name.
```

Listing 2. Codereader Configuration File (`codereader.yml`) — Part 2

```yaml
models:
  - name: Deepseek
    model: deepSeek-coder:6.7b
    runner: ollama
    runner_config:
      ollama_bin: ollama
    weight: 1.4
    enable: true

  - name: Phi
    model: phi3:medium
    runner: ollama
    runner_config:
      ollama_bin: ollama
    weight: 0.75
    enable: true

  - name: Qwen3.5
    model: qwen3.5:9b
    runner: ollama
    runner_config:
      ollama_bin: ollama
    weight: 1.25
    enable: true
```

Listing 3. Codereader Configuration File (`codereader.yml`) — Part 3

Appendix D: Java Test Examples

In this section we show a couple of examples of how these test cases look like when auto-generated.

Appendix D.1: EvoSuite

```java
@Test(timeout = 4000)
public void test42() throws Throwable {
    Calculator calculator0 = new Calculator();
      double double0 = calculator0.modulo(1.0,
1408.0511499512309);
                                  assertEquals(1,
calculator0.getOperationCount());
    assertEquals(1.0, double0, 0.01);
}
```

Listing 4. Example of an EvoSuite-generated test before renaming

Appendix D.2: Randoop

```java
@Test
public void test011() throws Throwable {
    if (debug)
                        System.out.format("%n%s%n",
"RegressionTest0.test011");
    Calculator calculator0 = new Calculator();
    calculator0.reset();
                  java.lang.String     str3     =
calculator0.classify((double) (byte) 10);
    calculator0.memoryStore(0.0d);
    calculator0.memoryClear();
      double double9 = calculator0.max((double)
10.0f, (-1.0d));
          java.lang.Class<?>  wildcardClass10  =
calculator0.getClass();
     org.junit.Assert.assertEquals("'" + str3 +
"' != '" + "medium" + "'", str3, "medium");
     org.junit.Assert.assertTrue("'" + double9 +
"' != '" + 10.0d + "'", double9 == 10.0d);
    org.junit.Assert.assertNotNull(
          wildcardClass10
        );
}
```

Listing 5. Example of a Randoop-generated test before renaming

Appendix D.3: Obfuscated code

```java
public class TestClass10000 {
  @Test public void func_1() {
    Map<Integer, String> var_1 =
      new HashMap<Integer, String>();
    var_1.put(1, "one");
    var_1.put(2, "two");
    var_1.put(3, "three");
    var_1.put(4, "four");
    Assert.assertEquals(
      "{ 1 -> one , 2 -> two ,
       3 -> three , 4 -> four }",
      MapUtils.toString(var_1));
  }
}
```

Listing 6. An example of obfuscated test case

Appendix D.4: JSONL file

```
{
     "prompt": "public class TestClass10000
{\n@Test public void func_1() { Map<Integer,
String> var_1 = new HashMap<Integer, String>();
var_1.put(1, \"one\"); var_1.put(2, \"two\");
var_1.put(3, \"three\"); var_1.put(4, \"four\");
Assert.assertEquals(\"{ 1 -> one , 2 -
> two , 3 -> three , 4 -> four }\",
MapUtils.toString(var_1)); }\n}",
     "response": "public class TestClass10000
{\n@Test public void shouldPrettyPrintMap()
{ Map<Integer, String> map = new HashMap<Integer,
String>(); map.put(1, \"one\"); map.put(2,
\"two\"); map.put(3, \"three\"); map.put(4,
\"four\"); Assert.assertEquals(\"{ 1 -> one ,
2 -> two , 3 -> three , 4 -> four }\",
MapUtils.toString(map)); }\n}"
}
```

Listing 7. An example Jsonl dataset sample

Appendix E: Sample Size Justification

This appendix provides the statistical justification for the choice of 100 evaluation samples per checkpoint, as referenced in the Dataset section.

Appendix E.1: Method

Effect sizes are computed using the Wilcoxon signed-rank test on paired observations, following the non-parametric approach recommended for software engineering experiments with non-normal distributions [31]. The effect-size statistic is $r = \sqrt{\frac{Z}{N}}$, interpreted as small ($|r| \geq 0.10$), medium ($|r| \geq 0.30$), or large ($|r| \geq 0.50$) following Cohen [37]. To correct for small-sample inflation, r is bias-adjusted using the Olkin–Pratt estimator [38]. The 95% confidence interval on r is obtained via the Fisher Z-transformation [39]. The recommended sample size n is derived from the conservative lower bound of this confidence interval, converted to Cohen's d and passed through the standard formula for a two-sided test at $\alpha = 0.05$, power $= 0.80$, with a 25% non-normality correction applied to account for the Wilcoxon efficiency ratio relative to a t-test.

The pilot run used to estimate effect sizes consisted of 30 randomly sampled test methods evaluated across all six Qwen2.5-Coder checkpoints. Effect sizes from this pilot are conservative inputs to the power analysis: using the lower bound of the 95% confidence interval on r ensures that the recommended n accounts for uncertainty in the pilot estimate itself.

Appendix E.2: Primary comparisons

Table IV reports the recommended n for all comparisons that directly answer the research questions. All effects are large and highly significant ($p < 0.001$); the recommended n never exceeds 10 across any checkpoint.

TABLE IV
Recommended n for primary comparisons across fine-tuning checkpoints. All effects are large ($|r| \geq 0.50$, $p < 0.001$).

Comparison	0%	15%	25%	50%	75%	100%
F1 vs zero baseline	2	2	2	2	2	2
LLM avg: obf vs renamed	2	4	4	3	7	3
LLM wavg: obf vs renamed	3	5	5	4	10	3
LLM avg: obf vs oracle	2	3	2	3	3	2

Appendix E.3: Quality gap comparison

Three observations justify accepting 100 samples despite these large values for some conditions. First, the effects driving the large recommended n are small and statistically fragile: at 0%, the effect ($r = -0.13$, $p = 0.20$) is not significant at all, and at 50% ($r = -0.22$, $p = 0.027$) it is only marginally so. Second, this instability is itself a finding, as the base and partially fine-tuned models produce variable-quality output that is sometimes near-oracle and sometimes substantially below it, a characteristic that additional samples would not resolve. Third, the compute cost makes scaling infeasible, as shown in Table V.

TABLE V
Wall-clock runtime per 100-sample run and estimated time to reach the maximum recommended n of 2 085.

Models	Runtime (100 samples)	Est. for 2 085 samples
Qwen zero-shot	6.4 h	~133 h
Qwen 15%	5.3 h	—
Qwen 25%	4.5 h	—
Qwen 50%	4.8 h	—
Qwen 75%	5.7 h	—
Qwen 100%	3.3 h	—
GPT-4.1 mini	5.5	—
GPT-5 mini	6.22 h	—

Conclusions about the absolute quality gap between renamed and oracle output at underpowered checkpoints should therefore be treated as indicative rather than definitive.

Appendix F: Compute environment

All fine-tuning and inference experiments for Qwen2.5-Coder were conducted on cloud computing infrastructure (see Table VI), as full fine-tuning and checkpoint evaluation at the 7B parameter scale exceeded the capacity of available local hardware. The CodeReader judge ensemble was served together on the cloud via Ollama during evaluation.

TABLE VI
Compute environment details used on VastAI cloud services.

Component	Details
Cloud provider	VastAI [40]
GPU type	NVIDIA 5090
GPU memory	32 GB
Fine-tuning runs	6 checkpoints

All OpenAI evaluations had to be done on the local environment, because OpenAI blocks API calls from cloud providers. The local hardware environment is mentioned in Table VII.

TABLE VII
Compute environment details used on local environment.

Component	Details
GPU type	NVIDIA 3090
GPU memory	12 GB

Appendix G: Model usage statistics

Table VIII reports token usage and latency statistics for all Qwen2.5-Coder checkpoints, GPT-5 mini and GPT 4.1 mini.

A notable difference emerges between the two model families. Qwen2.5-Coder produces compact JSON completions averaging 43–48 tokens per file, with fine-tuned checkpoints generating slightly shorter responses than the untuned baseline (43–46 vs. 47.8 tokens), consistent with the model learning more concise naming patterns. Similar to Qwen2.5-Coder, GPT-4.1 mini produces short sets of tokens per file, on average 37 tokens. GPT-5 mini, by contrast, produces substantially more verbose completions averaging 562.6 tokens per file, roughly 12 times more than any Qwen checkpoint. Inspection of the raw responses reveals that GPT-5 mini does not confine itself to the requested JSON object but frequently includes reasoning, explanations, and commentary alongside the mapping, despite the prompt explicitly instructing JSON-only output.

GPT-5 mini averages 9,076 ms per file, which is comparable to Qwen's 3,000–10,000 ms range, though Qwen exhibits greater variance due to cloud infrastructure fluctuations. GPT-4.1 mini stands out with a markedly lower average latency of 1,333 ms, roughly seven times faster than GPT-5 mini. This gap is likely attributable to GPT-5 mini's internal reasoning mechanism, which incurs additional compute overhead before producing a response.

All models achieved a 100% success rate across 100 files.

TABLE VIII
Token usage and latency statistics per checkpoint. Prompt tokens are near-identical across all models as the same input files and prompt template were used.

Model	Prompt tokens	Completion tokens	Total tokens	Avg latency (ms)	Success
Qwen (0%)	53,053	4,779	57,832	2,940	100/100
Qwen (15%)	53,053	4,333	57,386	7,582	100/100
Qwen (25%)	53,053	4,589	57,642	8,274	100/100
Qwen (50%)	53,053	4,387	57,440	7,044	100/100
Qwen (75%)	53,053	4,488	57,541	9,943	100/100
Qwen (100%)	53,053	4,542	57,595	4,779	100/100
GPT-4.1 mini	53,343	3,708	57,051	1,333	100/100
GPT-5 mini	53,243	56,263	109,506	9,076	100/100

Appendix H: Model Selection: Qwen2.5-Coder

At the time of model selection in November 2025, the Qwen2.5-Coder family was the leading open-source code model series across its size range. The 7B variant was selected based on the hardware available at that stage of the project: a local setup on which larger variants were not practically deployable. The 14B model requires approximately 9 GB of VRAM for inference and the 32B model approximately 20 GB, both exceeding local capacity, while the 7B model (approx. 4.7 GB for inference) remained feasible. Fine-tuning was subsequently performed on a cloud environment, where even the 7B model required approximately 20 GB of VRAM under QLoRA, confirming that the larger variants would not have been viable under the original local constraints in any case. Full details of the compute environment used for fine-tuning are provided in Appendix F.

TABLE IX
Qwen2.5-Coder benchmark scores across model sizes at time of release (November 2024). Bold values indicate the best score per benchmark row.

Benchmark	32B	14B	7B	3B	1.5B	0.5B
HumanEval	**92.7**	89.6	88.4	84.1	70.7	61.6
MBPP	**90.2**	86.2	83.5	73.6	69.2	52.4
EvalPlus	**86.3**	84.0	81.9	75.2	66.5	53.8
MultiPL-E	79.4	**79.6**	76.5	72.1	56.7	49.6
McEval	**65.9**	61.9	60.3	42.6	33.8	18.7
LiveCodeBench	**31.4**	23.4	18.2	10.8	6.1	2.0
CRUXEval-O	**83.4**	79.5	65.9	56.0	37.5	27.8
BigCodeBench	**38.3**	36.3	29.8	29.4	37.6	34.5
Spider	**85.1**	84.8	83.1	76.8	68.8	55.7
BIRD-SQL	**58.4**	56.9	52.5	38.7	28.4	15.1
CodeArena	**68.9**	50.0	43.1	28.3	13.3	5.5

Appendix I: Model selection: Readability tool

DeepSeek-Coder 6.7B was preferred over its successors, because its successors consumed to much vram to be ran on the local hardware together with the other models. A quick estimate was that its direct successor (DeepSeek-Coder V2), whose smallest available variant has 16B parameters and requires approximately 10.4 GB of VRAM at Q4, leaving insufficient headroom for the KV cache on the available hardware and causing out-of-memory errors in practice [41].

Phi-3-medium was preferred over Phi-4 on hardware and task-fit grounds. Phi-4 at Q4 quantization requires approximately 10.5 GB of VRAM, leaving only approximately 1.5 GB for the KV cache, which caused out-of-memory errors during evaluation [42]. Phi-3-medium's strong instruction-following capability, evidenced by an MT-bench score of 8.9 out of 10 [43], makes it well suited to consistently applying a structured grading rubric across diverse test methods.

Qwen3.5-9B was selected as the third ensemble member over earlier Qwen variants on the basis of its improved general reasoning capabilities and compact hardware footprint of approximately 6 GB at Q4, leaving ample headroom within the 12 GB budget. Its inclusion provides complementary coverage of natural language criteria such as behavioral specificity, which are less central to a code-specialized model [44].

Appendix J: GGNN Graph Conversion

The GGNN technique does not operate directly on Java source files. Instead, each test method must first be converted into a `.proto` graph file using the `features-javac` plugin [5], which extracts an Abstract Syntax Tree augmented with dataflow edges from the Java source code. This conversion requires the plugin to perform compiler-like analysis on the code, meaning that all referenced types and methods must be resolvable.

Because our test methods are wrapped in dummy classes and all external references — such as imported types and called methods — are not available in the isolated snippet, the plugin throws resolution errors for these references. In most cases it still produces a valid `.proto` file despite these errors, but in some cases the conversion fails entirely. In our 100-sample evaluation subset, 5 out of 100 files failed conversion: `TestClass1404.java`, `TestClass24619.java`, `TestClass25504.java`, `TestClass36753.java`, and `TestClass69357.java`. For these files, the obfuscated identifiers are returned unchanged, directly contributing to a lower F1 score.

This is consistent with De Keersmaeker's observation that GGNN struggles when the method under test is absent from the graph, since the dataflow edges that connect test variables to the production method they exercise cannot be constructed [6]. The combination of failed conversions and the missing focal method context explains the gap between GGNN's F1 = 0.241 on our subset and the F1 = 0.296 reported by De Keersmaeker on his full dataset.